```
object Main {
 ...
 val sharding = ClusterSharding(system)                                    ❶
 val entityDefinition = Entity(SPContainer.typeKey)(entityContext =>
        SPContainer(entityContext.entityId)))

 val shardRegion: ActorRef[ShardingEnvelope[SPContainer.Command]] =        ❷
     sharding.init(entityDefinition)
 ...
}
```

❶ Creates a sharded cluster

❷ Creates the SR holding SPContainer

9.2.3 Available effects

Let's quickly run down the options regarding command handler effects. `Effect` is a factory of functions available to persistent actors that represents the different reactions to a `Command`. The functions are as follows:

- `persist`—Persists a single event or multiple events. There are several functions with this name that accept one or more events. This operation is performed atomically: all or none of the events are persisted.
- `none`—Does not persist the event. Not all commands produce an event, as in the example `GetCargos`.
- `unhandled`—Indicates that the command is intentionally not operational to avoid throwing an exception when such a command is received.

These three functions are mutually exclusive. You can add side effects to them with

```
thenRun(callback: State => Unit): EffectBuilder[Event, State]
```

to which you must pass the desired `callback`. The signature shows that this side effect can access the state and doesn't need to return anything. You can chain several side effects with the guarantee that their callbacks will be executed in order. However, effects appended to `persist` will not be executed if `persist` fails.

You can run `persist` after another effect, but doing so is not recommended. There is an important drawback: `persist` can always fail, and if that happens, the actor can signal with that other effect something that didn't occur. For example, the persistent entity could send a message to another actor, notifying it about `AddedCargo`, and then persist it. In doing so, the notified actor assumes that the cargo was added to the container—but if persistence fails, the assumption is false.